

Kapıdaki Güvenlik Görevlisi: Tarayıcı vs. Sunucu

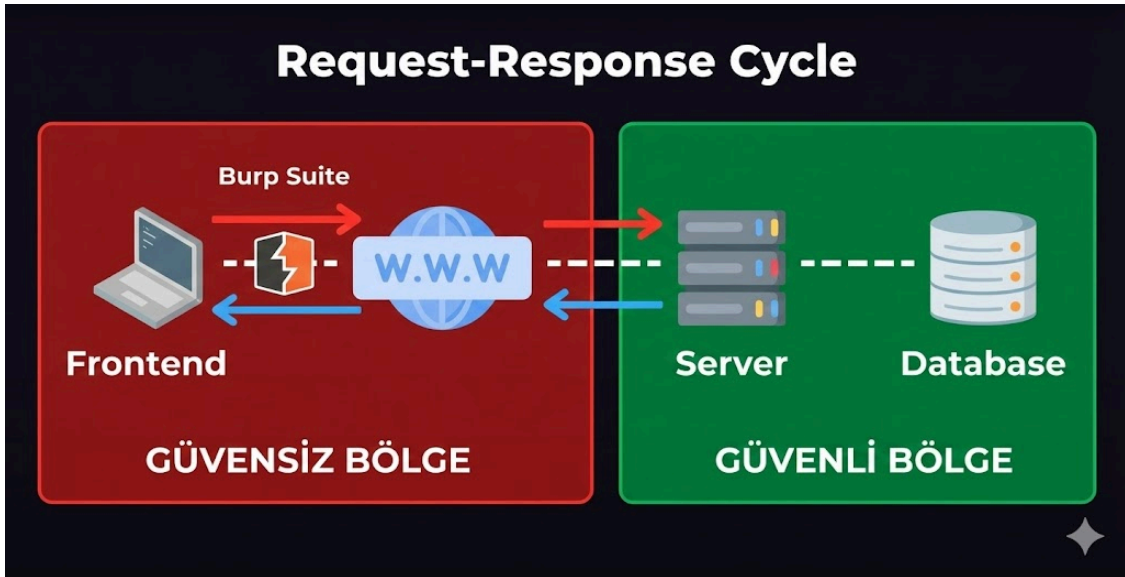
Güvenliğin Görünen ve Görünmeyen Yüzü:

Web uygulamalarında güvenlik çoğu zaman tarayıcı ekranında başlar gibi görünür. Formlar, uyarılar ve kısıtlamalar kullanıcıya bir güven hissi verir. Ancak bu hissin büyük bir kısmı bir **illüzyondur**. Gerçek güvenlik, tarayıcı ile sunucu arasında gerçekleşen HTTP iletişiminin doğru anlaşılmasına ve kontrol edilmesine dayanır. Bu yazıda, HTTP'nin yapısal özelliklerinin nasıl güvenlik açıklarına yol açabildiğini ve bu açıkların hangi mekanizmalarla kapatılmaya çalışıldığını ele alacağım.

1. Request-Response Döngüsü: Web'in Temel Mekanizması

Bir web uygulamasının en temel çalışma modeli **request-response (istek-yanıt)** döngüsüdür.

Basitleştirilmiş hâliyle süreç şu şekildedir:



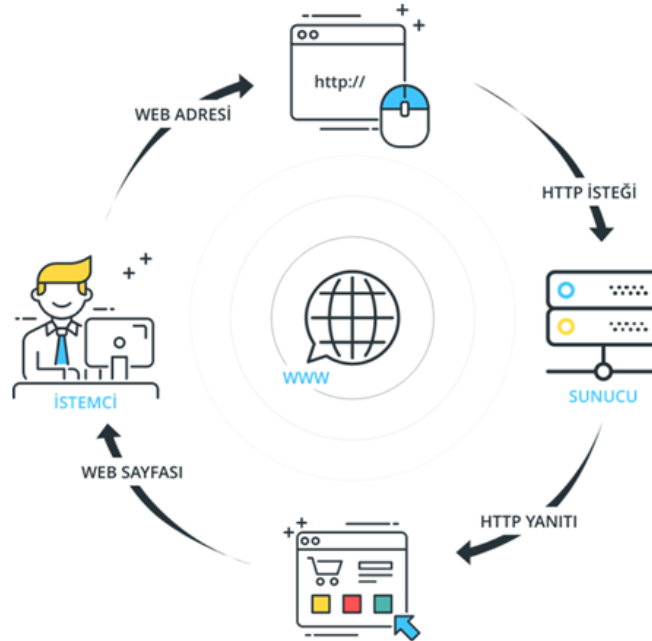
Tarayıcı (Client)
↓ HTTP Request
Sunucu (Server)
↑ HTTP Response
Tarayıcı (Client)

Tarayıcı, sunucuya bir istek gönderir (örneğin bir login formu). Sunucu bu isteği işler ve bir cevap döndürür. Bu döngü her işlemde yeniden başlar.

Kritik nokta şudur:

Sunucu, tarayıcının **nasıl bir arayüz sunduğunu değil**, kendisine gelen **ham HTTP isteğini** dikkate alır.

2. HTTP'nin "Stateless" Yapısı ve Güvenlik Çıkmazı



HTTP, **stateless** bir protokoldür. Bunun anlamı şudur:

Sunucu, gönderilen her isteği **bir öncekinden tamamen bağımsız** olarak değerlendirir. Yani teknik olarak sunucu, bir önceki istekte kimin giriş yaptığını hatırlamaz.

Bu durum řu soruyu aklımıza getirir:

| Bir alışveriş sitesi, kullanıcının hâlâ giriş yapmış halde olduğunu nasıl bilir?

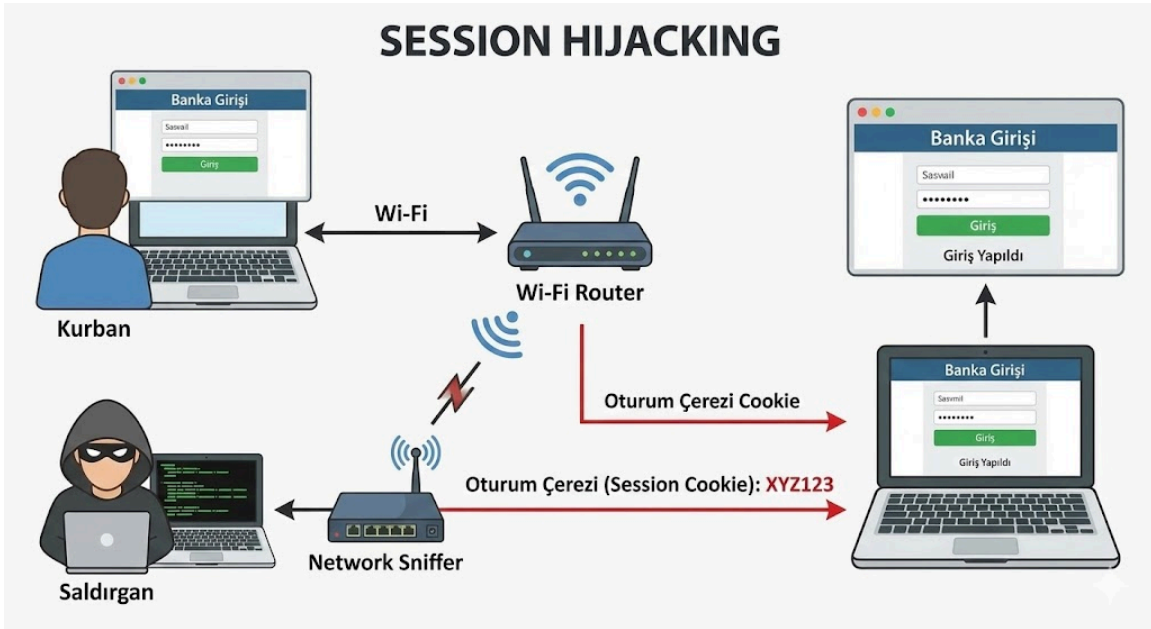
Cookie ve Session Mekanizması

Bu problem, **session** ve **cookie** mekanizmalarıyla aşılır. Kullanıcı başarılı bir şekilde giriş yaptığında:

- Sunucu, kullanıcıya özel bir **Session ID** üretir.
- Bu ID, tarayıcıya **cookie** olarak gönderilir.
- Tarayıcı, yaptığı her yeni istekte bu cookie'yi otomatik olarak sunucuya ekler.
- Sunucu, bu ID'yi tanıyarak kullanıcının kim olduğunu "hatırlamış" olur.

Aslında hatırlanan şey kullanıcı değil, **session kimliğidir**.

Güvenlik Riski: Session Hijacking



Session Hijacking Neden Tehlikelidir?

Session hijacking, kullanıcı ile web uygulaması arasındaki oturumun saldırgan tarafından ele geçirilmesidir. Kullanıcı giriş yaptıktan sonra sistem, kimliği

doğrulamak için bir session ID/token kullanır. Bu token çalındığında saldırgan, kullanıcının kimliğine bürünebilir

Eğer bir saldırgan bu cookie'yi ele geçirirse:

- Kullanıcının şifresine ihtiyaç duymaz
- Doğrudan geçerli bir oturum üzerinden işlem yapabilir

Bu nedenle cookie'ler genellikle şu bayraklarla korunur:

- **HttpOnly**: JavaScript tarafından okunmasını engeller
- **Secure**: Sadece HTTPS üzerinden gönderilmesini sağlar
- **SameSite**: Üçüncü taraf isteklerde kullanılmasını sınırlar

Bu önlemler, HTTP'nin durumsuz yapısından kaynaklanan riskleri azaltmayı amaçlar.

3. HTTP Metotları ve Bilgi Sızıntısı

HTTP'de en sık kullanılan iki metot **GET** ve **POST**'tur.

- **GET** isteklerinde parametreler URL içerisinde taşınır
- **POST** isteklerinde veriler request body içinde gönderilir

Login Formunda GET Kullanmak Neden Tehlikelidir?

Bir login formunun GET metodu ile çalıştığını düşünelim:

```
/login?username=admin&password=123456
```

Bu durumda şifre:

- Tarayıcı geçmişine kaydedilir
- Sunucu ve proxy log'larında açıkça görünür
- Ağ dinleme (sniffing) sırasında okunabilir

Bu, kimlik bilgilerinin **açık metin olarak sızması** anlamına gelir.

Açık Metin Bilginin Riskleri

- Aynı şifrenin başka sistemlerde de kullanılması
- Log dosyalarına yetkisiz erişim
- Ortak bilgisayarlarda tarayıcı geçmişi üzerinden bilgi sızması

Bu yüzden login gibi hassas işlemlerde **POST** kullanımı **zorunluluk**, **GET** kullanımı ise **ciddi bir hatadır**.

4. Status Code'lar: Sistemin Konuşan Dili

HTTP status code'lar, sunucunun istemciye verdiği geri bildirimlerdir.

Bazı temel örnekler:

- **200 OK** – İstek başarılı, Her şey yolunda
- **301 / 302** – Bir sayfadan diğer bir sayfaya yönlendirme
- **403 Forbidden** – Erişim Yasak
- **404 Not Found** – Kaynak bulunamadı
- **500 Internal Server Error** – Sunucu hatası

Status Code'lar Nasıl Bilgi Sızdırır?

Bir login sayfasında:

- "Kullanıcı adı hatalı" → **404**
- "Şifre hatalı" → **403**

döndürülüyorsa, bu durum saldırgana şu bilgiyi verir:

“Bu kullanıcı adı sistemde var.”

Bu fark, **username enumeration** saldırılarına zemin hazırlar.

Bu nedenle güvenli sistemlerde login hatalarında mümkün olduğunca aynı status code ve benzer hata mesajları döndürülmesi tercih edilir.

Status Code	Normal Anlamı	Saldırgan İçin İpucu
200 OK	İstek başarılı	Yanlış input'a rağmen dönerse → veri sızıntısı

Status Code	Normal Anlamı	Saldırgan İçin İpucu
301/302	Yönlendirme	Gizli endpoint veya login mekanizması keşfi
403 Forbidden	Kaynak var ama erişim yok	Kaynağın varlığını doğrular
404 Not Found	Kaynak yok	Kullanıcı adı/dizin yokluğunu gösterir
500 Internal Server Error	Sunucu hatası	Stack trace, SQL hatası → mimari bilgisi

5. Tarayıcı (Frontend): Kapıdaki Güvenlik Görevlisi

Tarayıcı tarafında yapılan kontroller genellikle **frontend validation** olarak adlandırılır.

Örneğin:

- Maksimum karakter sınırı
- Sadece sayı kabul eden input
- Zorunlu alan uyarıları

Bu kontroller genellikle JavaScript ile uygulanır ve kullanıcı deneyimini iyileştirir, ancak **gerçek bir güvenlik önlemi değildir**.

Frontend Kısıtlamaları Nasıl Bypass Edilir?

Burp Suite gibi bir proxy aracı kullanıldığında:

- Tarayıcı ile sunucu arasındaki HTTP istekleri yakalanabilir
- Request body manuel olarak değiştirilebilir
- Tarayıcının izin vermediği değerler sunucuya gönderilebilir

Frontend kısıtlamaları (ör. max 10 karakter) → Burp ile değiştirilebilir.

Bu durum, şu temel gerçeği ortaya koyar:

Frontend'de yapılan hiçbir doğrulama, gerçek bir güvenlik önlemi değildir.

Frontend, kapıdaki güvenlik görevlisidir; ancak asıl kontrol, kapının arkasındaki sistemdedir.

6. Sunucu (Backend): Asıl Güvenliğin Sağlandığı Yer

Gerçek güvenlik:

- **Backend validation**
- **Input sanitization**
- **Prepared statements**

ile sağlanır.

Sunucu:

- Gelen her veriyi **şüpheli** kabul etmeli
- Tür, uzunluk ve format kontrolü yapmalı
- Veritabanı sorgularında kullanıcı girdisini doğrudan kullanmamalıdır

Asıl Kontrol Backend'de Olmalı aksi hâlde frontend ne kadar "güzel" olursa olsun, tek bir manipüle edilmiş request tüm sistemi riske atabilir.

Zilin Üstüne Bant Yapıştırmak (Frontend Doğrulama)

Tarayıcı tarafında (Client-side) yapılan JavaScript kontrolleri (örneğin; "Şifreniz en az 8 karakter olmalı" veya "Sadece sayı girebilirsiniz" uyarıları), aslında kapı ziline bant yapıştırmak kadar kolay aşılabılır engellerdir.

- **Teknik Detay:** Bir web sayfasındaki HTML veya JavaScript kodları tamamen kullanıcının bilgisayarına iner ve orada çalışır. Bu, kontrolün tamamen kullanıcıda (ve dolayısıyla saldırganı) olduğu anlamına gelir.
- **Burp Suite ile Görünmeyeni Görmek:** Bir saldırgan, tarayıcı ile sunucu arasına **Burp Suite** gibi bir Proxy aracı koyduğunda, tarayıcının "Bu veriyi gönderme" dediği her şeyi manipüle edebilir. Frontend'de `maxlength="10"` olan bir input alanına, Proxy üzerinden 1000 karakterlik zararlı bir SQL kodu enjekte edilebilir.

Riskler ve Gerçek Dünya Örnekleri

Frontend doğrulamalarına güvenmenin bedeli ağır olabilir.

- **Senaryo:** Bir e-ticaret sitesinde bir ürünün fiyatı 1000 TL olsun. HTML kodunda bu `<input type="hidden" value="1000">` olarak tutuluyorsa, saldırgan Burp Suite ile araya girip bu değeri "1" olarak değiştirip sunucuya gönderebilir. Eğer sunucu

(backend) fiyatı veritabanından tekrar kontrol etmezse, ürün 1 TL'ye satılmış olur.

- **Equifax Hack'i ve SQL Injection:** Tarihin en büyük veri sızıntılarından biri olan Equifax olayında veya klasik SQL Injection saldırılarında temel mantık benzerdir: Sunucu, gelen verinin "temiz" olduğunu varsayar. Saldırgan, frontend'i bypass ederek veritabanı sorgusunu manipüle eden karakterler (`' OR 1=1 --`) gönderir ve tüm veritabanını çeker.

Sonuç: Asıl Kontrol Backend'de Olmalı

Güvenlik, asla kullanıcının cihazında sağlanamaz.

- **Sanitization (Temizleme):** Gelen verideki zararlı karakterlerin temizlenmesi.
- **Validation (Doğrulama):** Verinin beklenen formatta olup olmadığının sunucu tarafında kontrol edilmesi.
- **Prepared Statements:** SQL Injection'ı önlemek için veritabanı sorgularının parametrelerle ayrıştırılması.

“Frontend kullanıcı deneyimi içindir, Backend güvenlik içindir.”

HTTP'nin durumsuz yapısı, metotların yanlış kullanımı, status code'ların kontrolsüz şekilde farklılaştırılması ve frontend'e duyulan kör güven, web uygulamalarında ciddi güvenlik açıklarına yol açar. Gerçek güvenlik, yalnızca sunucu tarafında yapılan doğrulamalar ve kontrollerle sağlanabilir.

Bir web uygulamasını güvenli kılan şey, kullanıcının ne gördüğü değil; **sunucunun gelen isteğe nasıl davrandığıdır.**